

 [learning-zone](#) / [docker-and-kubernetes-basics](#) Public[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)[docker-and-kubernetes-basics](#) / [docker-commands.md](#) **learning-zone** Update docker-commands.md 851ae22 · 5 years ago

662 lines (454 loc) · 14.8 KB

Docker Command Cheatsheet

[Install Docker Desktop on Windows](#)

Docker Login

Login to a registry:

```
docker login [OPTIONS] [SERVER]
```



[OPTIONS]:

-u/--username username

-p/--password password

Example:

```
1 docker login localhost:8080 // Login to a registry
```

[docker-and-kubernetes-basics](#) / [docker-commands.md](#)[↑ Top](#)

Preview

Code

Blame

Raw



Logout from a registry:

```
docker logout [SERVER]
```



Example:

```
docker logout localhost:8080 // Logout from a registry
```



[↑ back to top](#)

Docker Container Commands

Create a container:

```
docker container create [OPTIONS] IMAGE [COMMAND] 
```

Example:

```
docker container create -t -i sofyspace/hello-world
```



Rename an existing container:

```
docker container rename CONTAINER NEW_NAME 
```

Example:

```
docker container rename mssql sqlserver
```

Run a command in a new container:

```
docker container run [OPTIONS] IMAGE [COMMAND] [ARGUMENTS] 
```

Example:

```
docker container run -it --name sqlserver -d sofyp
```



Delete a container:

```
docker container rm [OPTIONS] CONTAINER [CONTAINER] 
```

Example:

```
docker container rm hello-world
```



Update the configuration of one or more containers:

```
docker container update [OPTIONS] CONTAINER [CONTAINER] 
```

Example:

```
docker container update --memory "1g" --cpuset-cpu
```



Start a container:

```
docker container start [OPTIONS] CONTAINER [CONTAINER...]
```

Example:

```
docker container start redis
```



Stop a running container:

```
docker container stop [OPTIONS] CONTAINER [CONTAINER...]
```

Example:

```
docker container stop redis
```

```
docker stop $(docker ps -a -q) // To stop all the
```



Stop a running container and start it up again:

```
docker container restart [OPTIONS] CONTAINER [CONTAINER...]
```

Example:

```
docker container restart redis
```



Pause processes in a running container:

```
docker container pause CONTAINER [CONTAINER...]
```

Example:

```
docker container pause redis
```

Unpause processes in a running container:

```
docker container unpause CONTAINER [CONTAINER...]
```

Example:

```
docker container unpause redis
```



Block a container until others stop (after which it prints their exit codes):

```
docker container wait CONTAINER [CONTAINER...]
```



Example:

```
docker container wait redis
```

Kill a container by sending a SIGKILL to a running container:

```
docker container kill [OPTIONS] CONTAINER [CONTAIN
```



Example:

```
docker container kill redis
```



Attach local standard input, output, and error streams to a running container:

```
docker container attach [OPTIONS] CONTAINER [CONTA
```



SSH into container:

```
docker exec -it [OPTIONS] CONTAINER [CONTAINER...]
```



Example

```
docker container exec -it redis /bin/sh
```



Check docker daemon disk space usage:

```
docker system df
```



Remove all dangling and unused images and containers:

```
docker system prune --all
```



[↑ back to top](#)

Docker Image Commands

Create an image from a Dockerfile:

```
docker build <Dockerfile_url>
```



docker build -t – builds an image from a Dockerfile in the current directory and tags the image

Pull an image from a registry:

```
docker image pull [OPTIONS] NAME[:TAG|@DIGEST]
```



Example:

```
docker image pull redis:latest
```

Push an image to a registry:

```
docker image push [OPTIONS] NAME[:TAG]
```



Example

```
docker image push sofyspace/redis:6.2
```

Create an image from a tarball:

```
docker import <URL/FILE>
```



Create an image from a container:

```
docker container commit <CONTAINER_NAME> <new_imag
```



Remove an image:

```
docker image rm <IMAGE_NAME>
```



Example

```
docker image rm -f redis
```

Load an image from a tar archive or stdin:

```
docker load <TAR_FILE/STDIN_FILE>
```



Save an image to a tar archive, streamed to STDOUT with all parent layers, tags, and versions:

```
docker save <IMAGE_NAME> > <TAR_FILE>
```



[↑ back to top](#)

Docker Commands for Container and Image Information

List running containers:

```
docker ps  
docker ps -a
```



`docker ps -a` – lists both running containers and ones that have stopped

List the logs from a running container:

```
docker container logs <CONTAINER_ID>  
  
# Example  
docker container logs busybox
```



List low-level information on Docker objects:

```
docker container inspect [OBJECT_NAME/ID]  
  
# Example  
docker container inspect busybox
```



List real-time events from a container:

```
docker events <CONTAINER_ID>
```



Show port (or specific) mapping for a container:

```
docker port <CONTAINER_ID>
```



Show running processes in a container:

```
docker top <CONTAINER_ID>
```



Show live resource usage statistics of containers:

```
docker stats <CONTAINER_ID>
```



Show changes to files (or directories) on a filesystem:

```
docker diff <CONTAINER_ID>
```



List all images that are locally stored with the docker engine:

```
docker image ls
```



Show the history of an image:

```
docker history [IMAGE]
```



[↑ back to top](#)

Networks

List networks:

```
docker network ls
```



Remove one or more networks:

```
docker network rm NETWORK [NETWORK...]
```



```
# Example:
```

```
docker network rm my-network
```

Inspect network:

```
docker network inspect networkname
```



```
# Example:
```

```
docker network inspect bridge
```

Connects a container to a network:

```
docker network connect [OPTIONS] NETWORK CONTAINER
```

Example:

```
docker network connect multi-host-network containe
```



Disconnect a container from a network:

```
docker network disconnect [OPTIONS] NETWORK CONTAI
```

Example:

```
docker network disconnect multi-host-network conta
```



Create network:

It is possible to create a network in Docker before launching containers

```
docker network create [OPTIONS] NETWORK
```

Example:

```
docker network create --driver bridge some_network
```



[↑ back to top](#)

Dockerfile

Dockerfile is a text document that contains set of commands and instructions which will be executed in a sequence in the docker environment for building a new docker image.

Command	Description
FROM	Sets the base image for subsequent
MAINTAINER	Sets the author field of the generated images
RUN	Execute commands in a new layer on top of the current image and commit the results

Command	Description
CMD	Allowed only once (if many then last one takes effect)
LABEL	Adds metadata to an image
EXPOSE	Informs container runtime that the container listens on the specified network ports at runtime
ENV	Sets an environment variable
ADD	Copy new files, directories, or remote file URLs from into the filesystem of the container
COPY	Copy new files or directories into the filesystem of the container
ENTRYPOINT	Allows you to configure a container that will run as an executable
VOLUME	Creates a mount point and marks it as holding externally mounted volumes from native host or other containers
USER	Sets the username or UID to use when running the image
WORKDIR	Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY, and ADD commands
ARG	Defines a variable that users can pass at build-time to the builder using --build-arg
ONBUILD	Adds an instruction to be executed later, when the image is used as the base for another build
STOPSIGNAL	Sets the system call signal that will be sent to the container to exit

FROM

This command sets the base image for subsequent instructions

```
FROM <image>
FROM <image>:<tag>
FROM <image>@<digest>
```



```
# Example:
FROM ubuntu
FROM ubuntu:latest
FROM ubuntu:18.04
```

RUN

RUN instruction allows you to install your application and packages required for it. It executes any commands on top of the current image and creates a new layer by committing the results. It is quite common to have multiple RUN instructions in a Dockerfile.

```
# Shell Form: RUN
RUN npm start
```



```
# Exec form RUN ["", "", ""]
RUN [ "npm", "start" ]
```

ENTRYPOINT

An ENTRYPOINT allows you to configure a container that will run as an executable. It is used to run when container starts.

```
# Exec Form:
ENTRYPOINT ["executable", "param1", "param2"]
```



```
#Shell Form:
ENTRYPOINT command param1 param2
```

```
# Example:
FROM alpine:3.5
ENTRYPOINT ["/bin/echo", "Print ENTRYPOINT instruc
```



If an image has an ENTRYPOINT and pass an argument to it while running the container, it won't override the existing entrypoint but it just appends what you passed with the entrypoint. To override the existing ENTRYPOINT, you should use `–entrypoint` flag for the running container.

```
# Build image:
docker build -t entryptImage .

# Run the image:
docker container run entryptImage // Print ENTR

# Override entrypt:
docker run --entrypt "/bin/echo" entryptImag
```



CMD

CMD instruction is used to set a default command, which will be executed only when you run a container without specifying a command. But if the docker container runs with a command, the default command will be ignored.

The CMD instruction has three forms,

```
# Exec form:
CMD ["executable","param1","param2"]

# Default params to ENTRYPOINT:
CMD ["param1","param2"]

# Shell form:
CMD command param1 param2
```

The main purpose of the CMD command is to launch the required software in a container. For example, running an executable .exe file or a Bash terminal as soon as the container starts.

Remember, if docker runs with executable and parameters then CMD instruction will be overridden(Unlike ENTRYPOINT).

```
docker run executable parameters
```

Note: There should only be one CMD command in your Dockerfile. Otherwise only the last instance of CMD will be executed.

COPY

The COPY instruction copies new files or directories from source and adds them to the destination filesystem of the container.

```
COPY [--chown=<user>:<group>] <src>... <dest>
COPY [--chown=<user>:<group>] ["<src>",... "<dest>"]

# Example:
COPY test.txt /absoluteDir/
COPY tes? /absoluteDir/          # Copies all files
```

The path must be relative to the source directory that is being built. Whereas is an absolute path, or a path relative to WORKDIR.

ADD

The ADD instruction copies new files, directories or remote file URLs from source and adds them to the filesystem of the image at the destination path. The functionality is similar to COPY command and supports two forms of usage,

```
ADD [--chown=<user>:<group>] <src>... <dest>
ADD [--chown=<user>:<group>] ["<src>",... "<dest>"]

# Example:
ADD test.txt /absoluteDir/
ADD tes? /absoluteDir/          # Copies all files o
```

ADD commands provides additional features such as downloading remote resources, extracting TAR files etc.

```
# Download an external file and copy to the destin
ADD http://source.file/url /destination/path

# Copies compressed files and extract the content
ADD source.file.tar.gz /temp
```

ENV

The ENV instruction sets the environment variable to the value . It has two forms,

The first form, `ENV <key> <value>`, will set a single variable to a value. The second form, `ENV <key>=<value> ...`, allows for multiple variables to be set at one time.

```
ENV <key> <value>
ENV <key>=<value> [<key>=<value> ...]
```



```
# Example:
ENV name="Chand Anima" age=30
ENV name Chand Anima
ENV age 30
```

EXPOSE

The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime. i.e, It helps in inter-container communication. You can specify whether the port listens on TCP or UDP, and the default is TCP.

```
EXPOSE <port> [<port>/<protocol>...]
```



```
# Example:
EXPOSE 80/udp
EXPOSE 80/tcp
```

But if you want to bind the port of the container with the host machine on which the container is running, use `-p` option of docker run command.

```
docker run -p <HOST_PORT>:<CONTAINER:PORT> IMAGE_N
```



```
# Example:
docker run -p 80:80/udp myDocker
```



WORKDIR

The WORKDIR command is used to define the working directory of a Docker container at any given time for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions that follow it in the Dockerfile.

```
WORKDIR /path/to/workdir
```



```
# Example:
```

```
WORKDIR /c
WORKDIR d
WORKDIR e
RUN pwd // /c/d/e
```

LABEL

The LABEL instruction adds metadata as key-value pairs to an image. Labels included in base or parent images (images in the FROM line) are inherited by your image.

```
LABEL <key>=<value> <key>=<value> <key>=<value> ..
```

Example:

```
LABEL version="1.0"
LABEL multi.label1="value1" \
      multi.label2="value2" \
      other="value3"
```



You can view an image's labels using the docker image inspect --format=' ' myimage command. The output would be as below,

```
{
  "version": "1.0",
  "multi.label1": "value1",
  "multi.label2": "value2",
  "other": "value3"
}
```

MAINTAINER

The MAINTAINER instruction sets the Author field of the generated images.

```
MAINTAINER <name>
```

Example:

```
MAINTAINER Gabby
```

This command is deprecated status now and the recommended usage is with LABEL command

```
LABEL maintainer="Gabby"
```

VOLUME

The VOLUME instruction creates a mount point with the specified name and mounted volumes from native host or other containers.

```
VOLUME ["/data"]
```



```
# Example:  
FROM ubuntu  
RUN mkdir /test  
VOLUME /test
```

Docker Compose

Docker Compose is a tool for running multi-container applications on Docker defined using the Compose file format.

Using Docker Compose is basically a three-step process:

1. Define your app's environment with a Dockerfile so it can be reproduced anywhere.
2. Define the services that make up your app in docker-compose.yml so they can be run together in an isolated environment.
3. Lastly, run `docker compose up` and Compose will start and run your entire app.

A `docker-compose.yml` looks like this:

```
version: "3.9" # optional since v1.27.0  
services:  
  web:  
    build: .  
    ports:  
      - "5000:5000"  
    volumes:  
      - ./code  
      - logvolume01:/var/log  
    links:  
      - redis  
  redis:  
    image: redis
```



```
volumes:  
  logvolume01: {}
```

[↑ back to top](#)